

Procura adversarial

Minimax, poda alfa-beta e avaliação de estados

Modelação • Algoritmos • Verificação



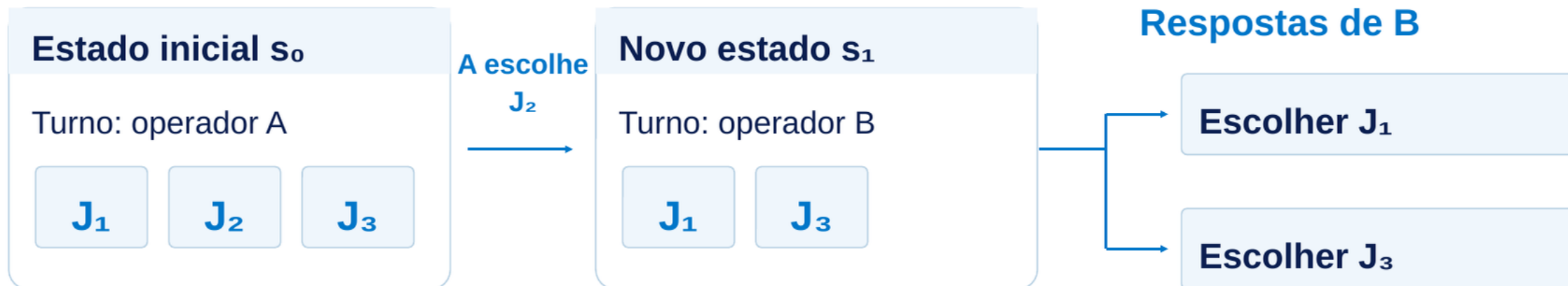
Janelas de atracação: decisões dependentes

Uma escolha altera as alternativas disponíveis para o outro operador.

Dois operadores: A e B

Escolhas alternadas

Atribuição exclusiva



$$J_{\text{livres}}(s_1) = J_{\text{livres}}(s_0) \setminus \{J_2\} = \{J_1, J_3\}$$

Que ação escolher, considerando as respostas que podem surgir?

Hipóteses do modelo adversarial

Distinguir a motivação operacional da simplificação usada no algoritmo.

Cenário operacional

Cada operador procura reduzir o próprio tempo de espera.

Interesses diferentes não demonstram, por si só, uma relação de soma zero.

Hipótese: $A = \text{MAX}$; $B = \text{MIN}$

MAX maximiza U ; MIN minimiza a mesma utilidade para MAX.

$$U_{\text{MIN}}(s) = -U_{\text{MAX}}(s)$$

Dois agentes

Turnos alternados

Informação completa

Transições determinísticas

Modelo didático de pior caso; não descreve automaticamente o comportamento real.

Representar o estado e a transição

Guardar a informação necessária para gerar ações e interpretar o resultado.

$$S = (J_{\text{livres}}, M_A, M_B, p)$$

Campos do estado

Campo	Significado
J livres	Janelas ainda disponíveis
M A	Janelas atribuídas a A
M B	Janelas atribuídas a B
p	Jogador do próximo turno

Exemplo: A escolhe J_2

ANTES

$(\{J_1, J_2, J_3\}, \emptyset, \emptyset, A)$



DEPOIS

$(\{J_1, J_3\}, \{J_2\}, \emptyset, B)$

J_2 sai das livres, entra em M A e o turno muda.

Uma transição atualiza as janelas, as atribuições e o próximo jogador.

Interface do jogo e condições de validade

Separar as regras do problema do algoritmo de procura.

Contrato das funções

Função	Responsabilidade
jogador(s)	Indica MAX ou MIN
acoes(s)	Devolve ações legais
resultado(s, a)	Cria o estado sucessor
terminal(s)	Sem janelas livres neste modelo
U(s)	Utilidade terminal para MAX

Verificações do modelo

A janela escolhida tem de estar livre.
O turno deve alternar entre A e B.
Os três conjuntos são disjuntos.

CONSERVAÇÃO DAS JANELAS

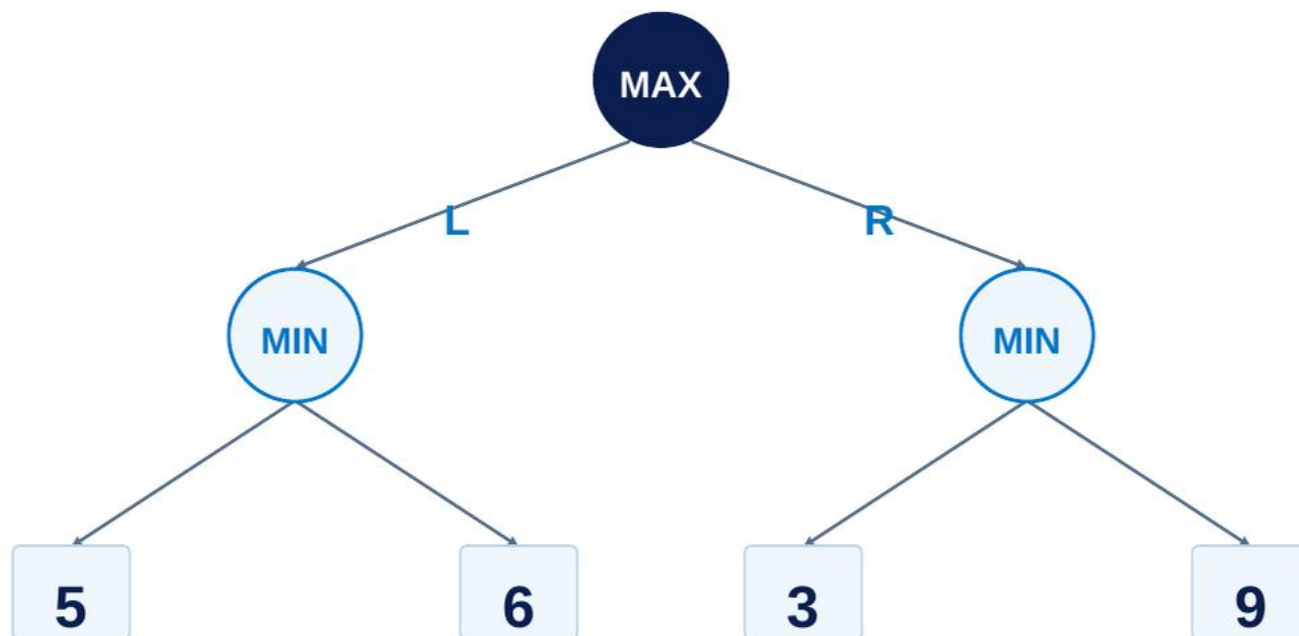
$$J_{\text{livres}} \cup M_A \cup M_B = J_0$$

J_0 : conjunto inicial. Verificações adicionais derivadas da representação proposta.

Explorar um ramo não pode alterar o estado usado pelos restantes ramos.

Da utilidade à árvore MAX/MIN

As ligações são ações; os valores das folhas são utilidades, não custos de arcos.



Uma escala para MAX

MAX escolhe valores maiores.
MIN escolhe valores menores.

Ação	Possível	Garantido
L	5 ou 6	5
R	3 ou 9	3

Nó = estado

Ligação = ação

Folha = U(s)

A folha 9 existe, mas MIN não é obrigado a escolher essa resposta.

Minimax: definição recursiva

Calcular um valor para cada estado e selecionar uma ação na raiz.

$$V(s) = \begin{cases} U(s) \\ \max_{a \in A(s)} V(\text{resultado}(s, a)) \\ \min_{a \in A(s)} V(\text{resultado}(s, a)) \end{cases}$$

se s é terminal

se s não é terminal e joga MAX

se s não é terminal e joga MIN

Escolha na raiz MAX

$$a^* = \arg \max_{a \in A(s)} V(\text{resultado}(s, a))$$

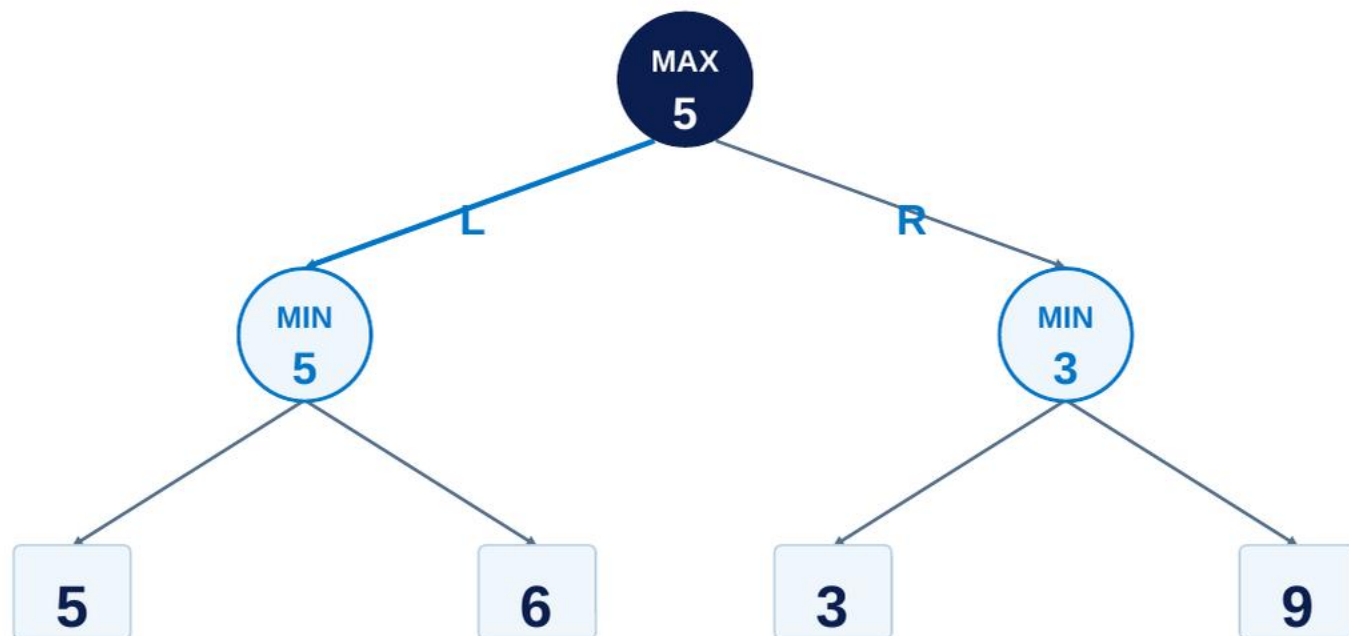
$A(s)$: ações legais em s .

Empate: conservar a primeira ação.

$V(s)$ é um valor; a^* é uma ação. Não são o mesmo objeto.

Minimax: execução completa

Propagar os resultados de baixo para cima, alternando mínimo e máximo.



Propagação dos valores

1 $V(L) = \min(5, 6) = 5$

2 $V(R) = \min(3, 9) = 3$

3 $V(s_0) = \max(5, 3) = 5$

4 folhas avaliadas • 7 nós visitados

A melhor ação é L: garante o valor 5 contra as respostas previstas no modelo.

Do valor à ação: pseudocódigo Minimax

A recursão calcula valores; a função da raiz guarda a ação correspondente.

Valor recursivo

```
01 VALOR(s):  
02   se terminal(s): devolver U(s)  
03   se jogador(s) = MAX:  $v \leftarrow -\infty$   
04   senão:  $v \leftarrow +\infty$   
05   para a em acoes(s):  
06      $r \leftarrow \text{VALOR}(\text{resultado}(s, a))$   
07   se jogador(s) = MAX:  
08      $v \leftarrow \max(v, r)$   
09   senão:  $v \leftarrow \min(v, r)$   
10   devolver v
```

Decisão na raiz MAX

```
01 DECIDIR(s):  
02   melhor  $\leftarrow -\infty$ ; escolha  $\leftarrow$  nenhuma  
03   para a em acoes(s):  
04      $r \leftarrow \text{VALOR}(\text{resultado}(s, a))$   
05     se escolha = nenhuma  
06       ou  $r > \text{melhor}$ :  
07       melhor  $\leftarrow r$ ; escolha  $\leftarrow a$   
08   devolver escolha, melhor
```

Pré-condição de DECIDIR:
s não terminal e jogador(s) = MAX.

Empate: atualizar a ação apenas perante uma melhoria estrita.

Complexidade: quanto custa explorar?

Um nível adicional pode multiplicar o trabalho por aproximadamente b .

Modelo de custo

b : fator de ramificação

d : profundidade em ações

$$N_{\text{folhas}} = b^d$$

$$N_{\text{total}} = \sum_{k=0}^d b^k$$

$$T_{\text{Minimax}} = O(b^d)$$

Exemplo: $b = 3$ e $d = 4$

Nível	Nós no nível	Acumulado
0	1	1
1	3	4
2	9	13
3	27	40
4	81	121

Árvore completa, com ramificação constante.
Contagens incluem a raiz no nível zero.

Medir separadamente folhas avaliadas, nós visitados e tempo de execução.

Por que podemos ignorar um ramo?

Uma informação parcial pode ser suficiente para excluir uma alternativa.

Alternativa já conhecida de MAX

$$V(L) = 5$$

Há uma ação que garante 5.

Novo ramo controlado por MIN

$$\min(3, x) \leq 3$$

Nenhum valor de x torna este ramo superior.

$$\max\{5, \min(3, x)\} = 5$$

x é a folha ainda não avaliada. Não precisamos de conhecer o seu valor.

A poda preserva o valor da árvore considerada; não é uma escolha aleatória.

Alfa e beta: transmissão, atualização e corte

Cada chamada recebe os limites úteis dos seus antecessores.

α / limite associado a MAX

Alternativa já encontrada para MAX
ao longo do caminho de pesquisa.

$$\alpha \leftarrow \max(\alpha, v)$$

Atualiza-se em MAX; não diminui.

β / limite associado a MIN

Alternativa já encontrada para MIN
ao longo do caminho de pesquisa.

$$\beta \leftarrow \min(\beta, v)$$

Atualiza-se em MIN; não aumenta.

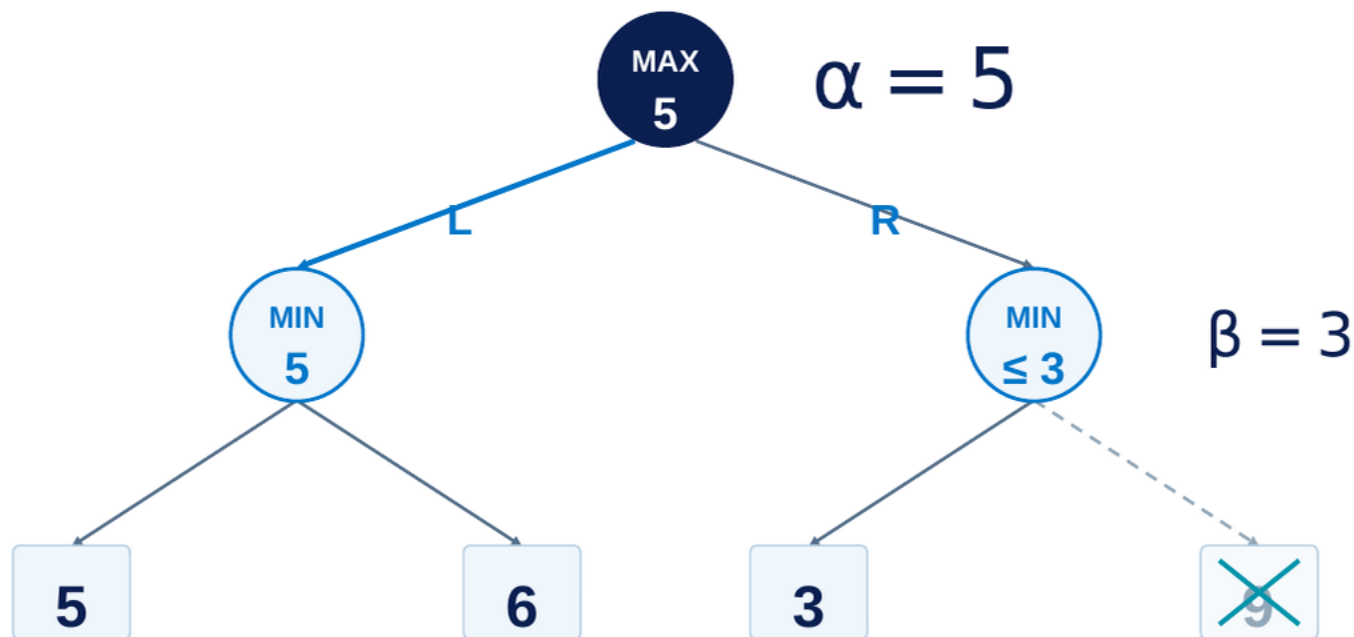
Na raiz $\alpha = -\infty, \quad \beta = +\infty$

$\alpha \geq \beta \Rightarrow \text{cortar}$

Os limites são locais a cada chamada, não variáveis globais entre ramos irmãos.

Alfa-beta: execução na árvore conhecida

Percorrer da esquerda para a direita; comparar com a execução completa.



Registo da execução

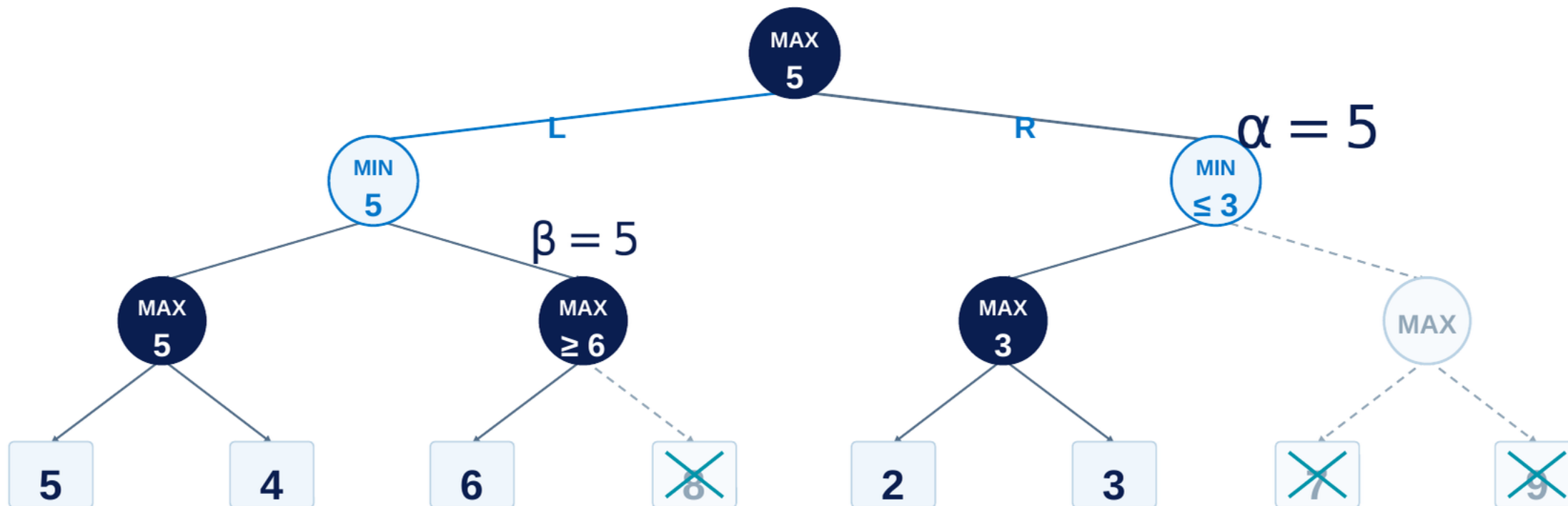
- 1 MIN esquerdo**
 $\min(5, 6) = 5$
- 2 Raiz MAX**
 α passa de $-\infty$ para 5
- 3 MIN direito**
Vê 3: $\beta = 3$; recebe $\alpha = 5$
- 4 Corte**
 $5 \geq 3$: a folha 9 não é avaliada

3 folhas avaliadas • 6 nós visitados

O valor da raiz continua a ser 5 e a ação escolhida continua a ser L.

Cortes em nós MAX e em nós MIN

Exemplo didático adicional: três níveis de decisões, da esquerda para a direita.



Corte em MAX: $6 \geq \beta = 5$

Corte em MIN: $\alpha = 5 \geq 3$

Minimax: 8 folhas / 15 nós → Alfa-beta: 5 folhas / 11 nós

Após o corte, o valor devolvido pode ser apenas um limite, não o valor exato do nó.

Pseudocódigo alfa-beta

A mesma recursão, com limites e uma condição de interrupção.

```
AB(s,  $\alpha$ ,  $\beta$ ): se terminal(s): devolver U(s)
```

Caso MAX

```
01 v ←  $-\infty$   
02 para a em acoes(s):  
03   r ← AB(resultado(s,a),  $\alpha$ ,  $\beta$ )  
04   v ← max(v, r)  
05    $\alpha$  ← max( $\alpha$ , v)  
06   se  $\alpha \geq \beta$ : interromper  
07 devolver v
```

Caso MIN

```
01 v ←  $+\infty$   
02 para a em acoes(s):  
03   r ← AB(resultado(s,a),  $\alpha$ ,  $\beta$ )  
04   v ← min(v, r)  
05    $\beta$  ← min( $\beta$ , v)  
06   se  $\alpha \geq \beta$ : interromper  
07 devolver v
```

Inicializar com AB(s_0 , $-\infty$, $+\infty$); guardar também a melhor ação na raiz.

Ordem de exploração e trabalho realizado

Manter a árvore e alterar apenas a ordem dos sucessores.

Algoritmo	Ordem proposta	Folhas	Nós	Valor / ação
Minimax	L: 5, 6 → R: 3, 9	4	7	5 / L
Alfa-beta	L: 5, 6 → R: 3, 9	3	6	5 / L
Alfa-beta	L: 5, 6 → R: 9, 3	4	7	5 / L
Alfa-beta	R: 3, 9 → L: 5, 6	4	7	5 / L

“Ordem proposta” inclui folhas que podem deixar de ser avaliadas por poda.

MAX: experimentar primeiro ações promissoras.

MIN: encontrar cedo respostas de valor baixo.

Boa ordenação pode poupar avaliações; o custo de ordenar também deve ser medido.

Profundidade limitada: terminal ou corte?

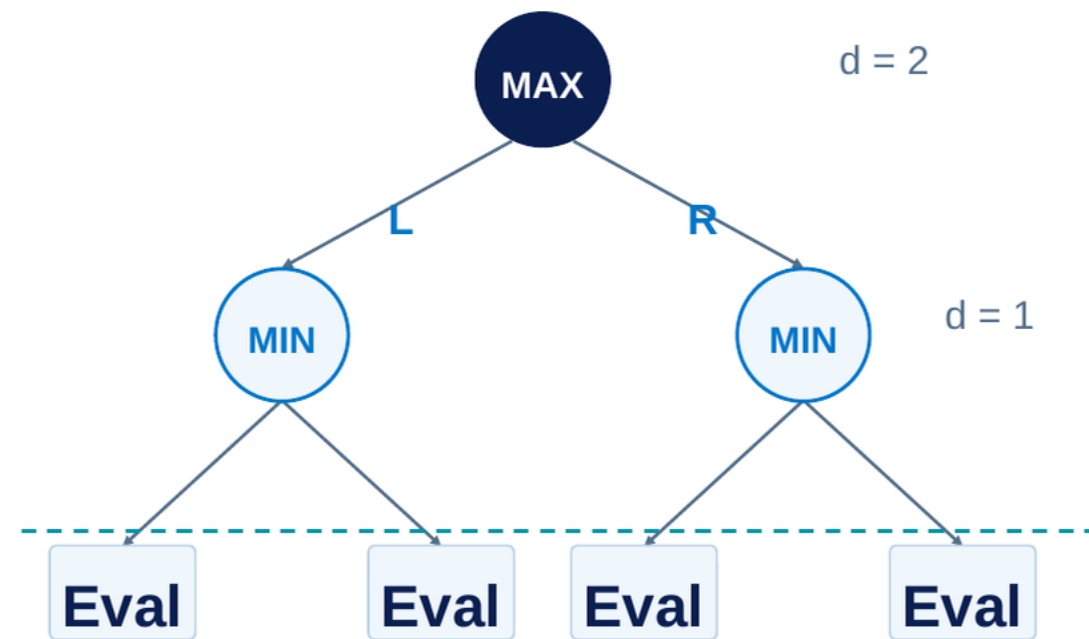
Uma ação de MAX ou MIN consome uma unidade de profundidade.

Duas condições de paragem

```
01 VALOR(s, d):  
02   se terminal(s): devolver U(s)  
03   se d = 0: devolver Eval(s)  
04   explorar filhos com d - 1  
05   agregar por máximo ou mínimo
```

d é o número de ações ainda a explorar.

O teste terminal vem antes do teste $d = 0$.



Limite: $d = 0$

Estados não terminais avaliados por estimativa.

A poda preserva o valor da árvore limitada; Eval aproxima o jogo completo.

Construir uma avaliação: sinais e escalas

Exemplo portuário proposto: espera e risco penalizam; capacidade beneficia MAX.

$$\text{Eval}(s) = -w_T \tilde{T}(s) - w_R \tilde{R}(s) + w_C \tilde{C}(s)$$

Espera: penalização

$$\tilde{T} = T/T_{\text{ref}}$$

$0 \leq T \leq T_{\text{ref}}$; $T_{\text{ref}} > 0$

Referência temporal fixa

Risco: penalização

$$\tilde{R} = R \in [0, 1]$$

R é uma probabilidade

Já está na escala $[0, 1]$

Capacidade: benefício

$$\tilde{C} = C/C_{\text{ref}}$$

$0 \leq C \leq C_{\text{ref}}$; $C_{\text{ref}} > 0$

Referência de capacidade fixa

Convenção didática: pesos não negativos, com soma 1.

Definir primeiro o significado e a escala dos indicadores; depois escolher os pesos.

Calcular a avaliação e analisar os pesos

Manter os estados; alterar apenas as prioridades expressas pelos pesos.

Indicadores normalizados

Estado	Espera $\tilde{T}$	Risco $\tilde{R}$	Capacidade $\tilde{C}$
S_1	0,2	0,4	0,8
S_2	0,5	0,2	0,6

$$\text{Eval}(s_1) = -0,5(0,2) - 0,3(0,4) + 0,2(0,8) = -0,06$$

Pesos (w_T, w_R, w_C)	$\text{Eval}(s_1)$	$\text{Eval}(s_2)$	Preferência
(0,5; 0,3; 0,2)	-0,06	-0,19	S_1
(0,2; 0,6; 0,2)	-0,12	-0,10	S_2

Uma alteração de pesos pode inverter a preferência. Calibrar Eval na escala de U.

Como verificar uma implementação?

Usar árvores pequenas com resultados conhecidos antes de medir desempenho.

Teste	Comportamento esperado
Estado inicial terminal	Devolver $U(s)$, sem sucessores
Ações empatadas	Conservar a primeira ação
$d = 0$, não terminal	Usar $Eval(s)$
Terminal no limite	Usar $U(s)$, não $Eval(s)$

Igualdade a verificar na raiz

$$V_{\alpha\beta}(s_0) = V_{\text{Minimax}}(s_0)$$

Limites iniciais: $(-\infty, +\infty)$.
Mesma árvore, profundidade
e função de avaliação.

Medições independentes

Folhas avaliadas

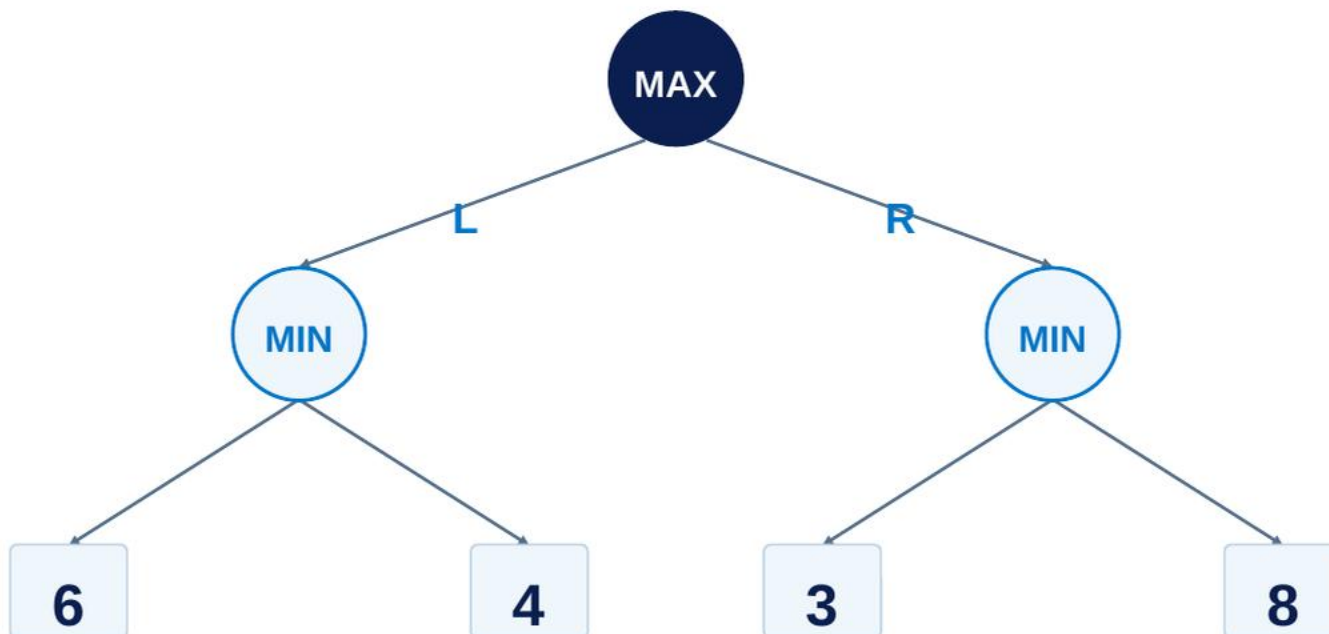
Nós visitados

Tempo de execução

Testar o valor na raiz; não exigir valores exatos em todos os nós podados.

Exercício integrado: calcular, podar e comparar

Percorrer da esquerda para a direita; iniciar $\alpha = -\infty$ e $\beta = +\infty$.



Quatro tarefas

- 1 Calcular os valores**
Valor da raiz e ação L ou R.
- 2 Registrar os limites**
Atualizações de α e β .
- 3 Justificar o corte**
Qual a folha não avaliada?
- 4 Comparar o trabalho**
Folhas e nós, com e sem poda.

Desafio: trocar a ordem 3, 8 por 8, 3. O que muda e o que se mantém?

Síntese: decisão, otimização e aproximação

Três componentes distintos de um agente de procura adversarial.

MINIMAX

Calcula o melhor valor contra a resposta de MIN.

Lógica da decisão

ALFA-BETA

Evita exploração que já não pode mudar a escolha.

Otimização da procura

LIMITE + EVAL

Estima estados ainda não terminais.

Aproximação ao jogo completo

Poda: preserva o valor da árvore considerada.

Corte heurístico: não garante o valor do jogo completo.

Representar → calcular → podar → avaliar → verificar.